INFORME TÉCNICO DE ARQUITECTURA E IMPLEMENTACIÓN

PROYECTO SISTEMAS OPERATIVOS - FASE I: SIMULADOR DE KERNEL

Asignatura:	Sistemas Operativos (UCAB)	**Profesor:**	Marcel J. Castro G.
Integrantes:	Emanuel Siso | Daniela Goncalves | Christian Vilera		
Fecha:	Noviembre 2026	**Estado:**	Entregable Oficial - Fase I Completa

1. RESUMEN EJECUTIVO Y ARQUITECTURA GENERAL

El presente proyecto implementa la Fase I de la arquitectura de un simulador de Kernel y gestor de procesos en consola. El sistema fue desarrollado bajo el estándar C++17 empleando un esquema desacoplado de concurrencia mediante hilos (Threads) y cerrojos de exclusión mutua (Mutex). La arquitectura separa estrictamente la Interfaz de Línea de Comandos (CLI Thread), encargada de la interacción con el usuario y parseo de comandos, del Motor de CPU (CPU Engine Thread), responsable de la simulación del reloj del sistema, la gestión del estado de los procesos y la ejecución de los algoritmos de planificación.

2. ESTRUCTURA Y DISEÑO MODULAR

El código fuente se organizó modularmente respetando la separación entre archivos de cabecera (.hpp) e implementación (.cpp):

- pcb.hpp / pcb.cpp: Modela la estructura del Process Control Block (PCB) conteniendo el identificador único (PID), nombre, estado (NEW, READY, RUNNING, PAUSED, TERMINATED), ráfaga inicial y restante, prioridad (1-5), y las métricas de tiempo (WT, TAT, RT).
- scheduler.hpp / scheduler.cpp: Implementa la clase Scheduler que administra la cola de procesos, el ciclo de CPU en un hilo independiente, y las políticas de planificación Round Robin (RR) y Shortest Remaining Time First (SRTF).
- cli.hpp / cli.cpp: Modela la interfaz de consola interactiva con validación estricta de parámetros para prevenir caídas por datos inválidos.
- main.cpp: Punto de entrada que coordina el ciclo de vida del simulador.

3. MECANISMOS DE CONCURRENCIA Y SINCRONIZACIÓN

Para evitar condiciones de carrera (Race Conditions) sobre las estructuras de datos compartidas (all_processes, pending_arrivals, gantt_log), se implementó un envoltorio de exclusión mutua (PlatformMutex) compatible con Win32 API (CRITICAL_SECTION) y POSIX (pthread_mutex_t). Cada acceso a la lista de procesos desde la CLI (comandos create, load, ps, pcb, stats) o desde el motor de CPU se encuentra rigurosamente protegido mediante el patrón RAII (PlatformMutexGuard), garantizando la integridad de la memoria.

4. ALGORITMOS DE PLANIFICACIÓN Y MÉTRICAS

1. Round Robin (RR): Asigna un quantum fijo de tiempo a cada proceso en estado READY. Al expirar el quantum, el proceso es desapropiado y colocado nuevamente en la cola lógica de listos.
2. Shortest Remaining Time First (SRTF): Algoritmo apropiativo que evalúa en cada tick de reloj si ha arribado o cambiado un proceso con menor ráfaga restante (remaining_burst) que el proceso en ejecución, realizando la desapropiación inmediata.

Métricas calculadas: Tiempo de Espera (WT = TAT - BT), Tiempo de Retorno (TAT = CT - AT), Tiempo de Respuesta (RT = ST - AT) y % de Uso de CPU.

ANEXO OBLIGATORIO: DECLARACIÓN ÉTICA DE USO DE INTELIGENCIA ARTIFICIAL

Los integrantes Emanuel Siso, Daniela Goncalves y Christian Vilera declaramos que en la elaboración de esta Fase I del Proyecto se emplearon herramientas de Asistencia por IA (Gemini / LLM) como apoyo para la estructuración de clases C++, diseño de patrones de sincronización multihilo y generación de documentación. Todo el código fue revisado, depurado, adaptado y verificado manualmente por el equipo para garantizar su correcto funcionamiento, cumplimiento de especificaciones académicas y autoría intelectual del proyecto.